

DATA 110 • FALL 2026

Visualizing distributions

Rebin Muhammad

Montgomery College

The argument follows Claus O. Wilke, *Fundamentals of Data Visualization*, Chapter 7. The code in this lecture is Python, using pandas, numpy, matplotlib, and scikit-learn.

TODAY

How the ages of 756 people are spread out

- A distribution is the pattern of the values: where they pile up, and where they are rare.
- A histogram makes that pattern by counting. You choose the bins.
- A density curve makes the pattern by smoothing. You choose how wide the smooth is.

The example is passenger age on the Titanic. After the pictures, the same choices are written in matplotlib.

PART 1

Histograms

Choose the edges. Count how many cases fall between them. The height of the bar is that count.

One row is one passenger

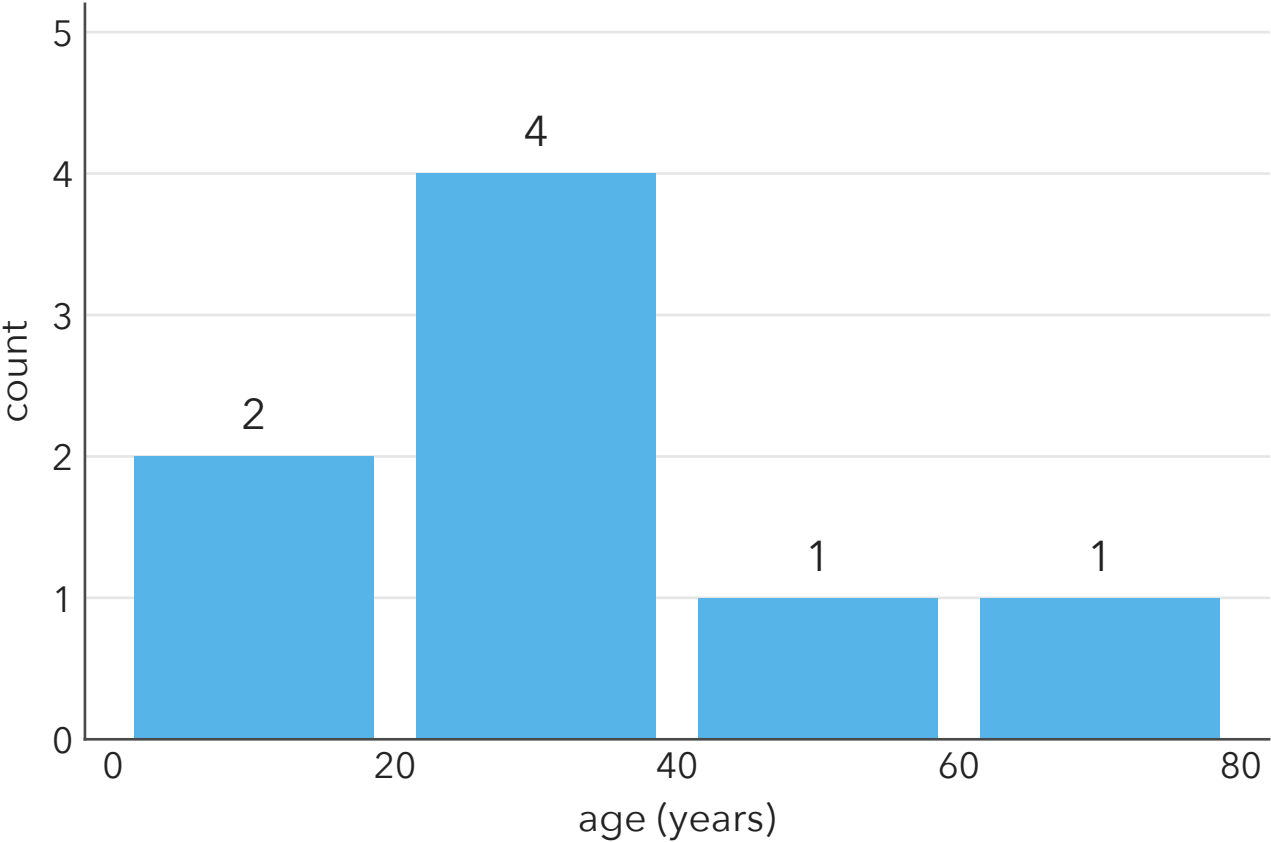
The table has 756 passengers. Age is in years, so 0.17 is about two months. The oldest passenger in this table is 71. These are the six youngest, sorted by age.

AGE	SEX	CLASS	SURVIVED
0.17	female	3rd	survived
0.33	male	3rd	died
0.80	male	2nd	survived
0.83	male	2nd	survived
0.83	male	3rd	survived
0.92	male	1st	survived

Eight ages, counted by hand

These eight ages are a made-up example. The height of the bar from a to b is the number of ages with $a \leq \text{age} < b$. The 756 passengers are the next slide.

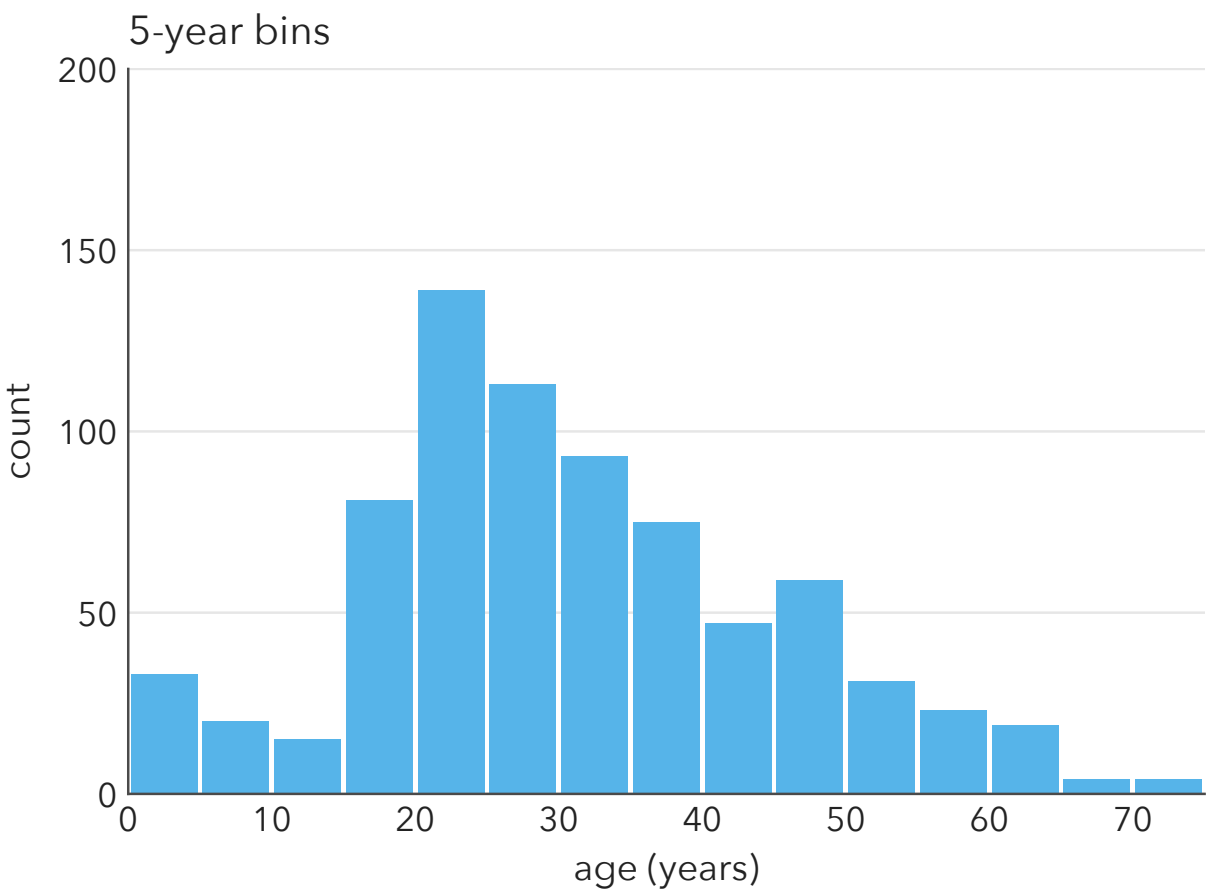
BIN	AGES	COUNT
0-20	3, 8	2
20-40	21, 22, 24, 31	4
40-60	48	1
60-80	67	1
Total		8



Five-year bins, then the counts

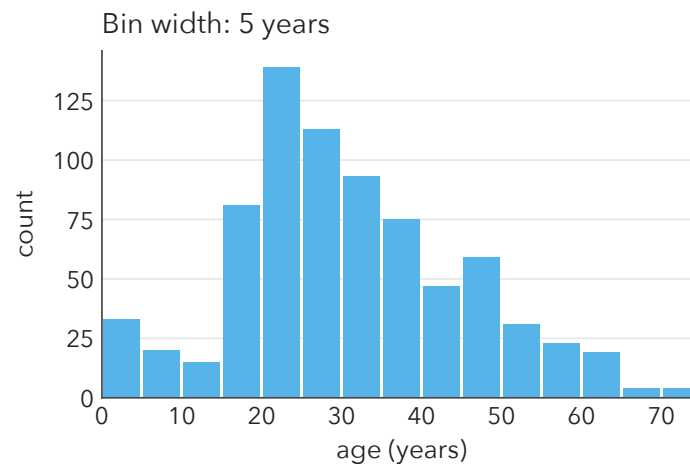
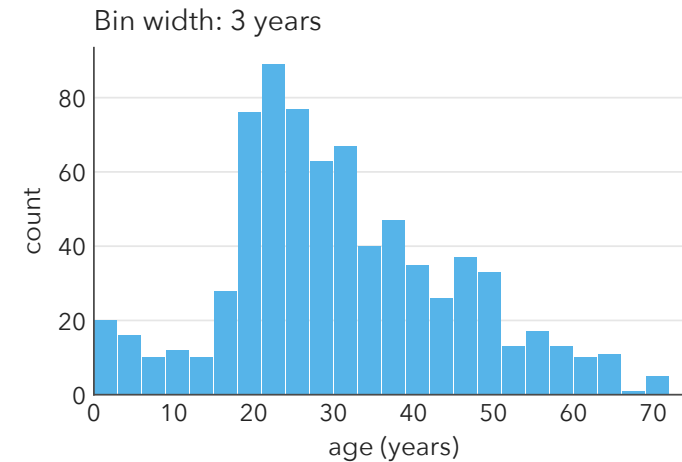
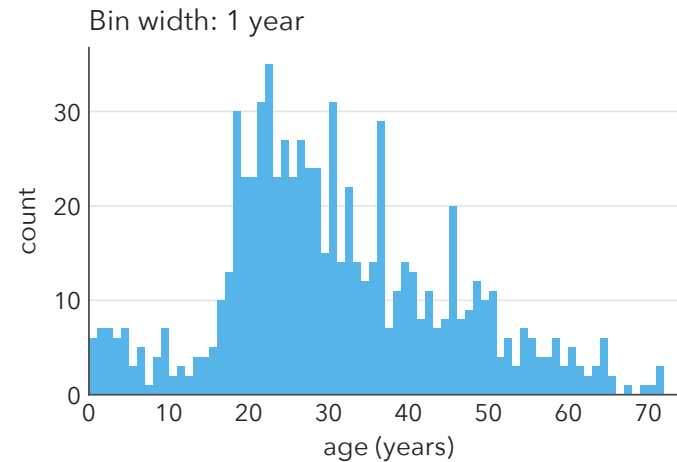
Each bar covers five years: 0–5, 5–10, and so on. The tallest bar is ages 20–25, with 139 passengers. The bars start at zero because the height is a count. An age of exactly 25 is counted in 25–30: a bin includes its left edge and stops before its right edge.

AGE	COUNT	AGE	COUNT
0–5	33	40–45	47
5–10	20	45–50	59
10–15	15	50–55	31
15–20	81	55–60	23
20–25	139	60–65	19
25–30	113	65–70	4
30–35	93	70–75	4
35–40	75	75–80	0



The same passengers, four bin widths

Nothing in the table changed. Only the edges did. A width of one year keeps every small bump. A width of 15 years blends the children into the adults, and the peak near age 20 becomes a blunt step.



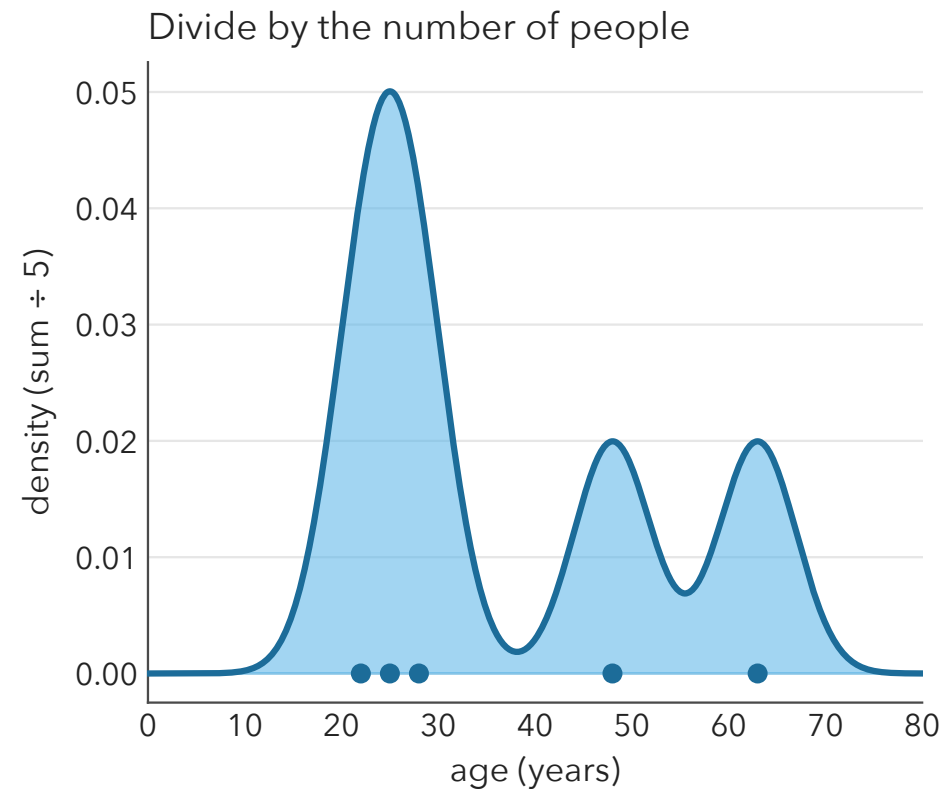
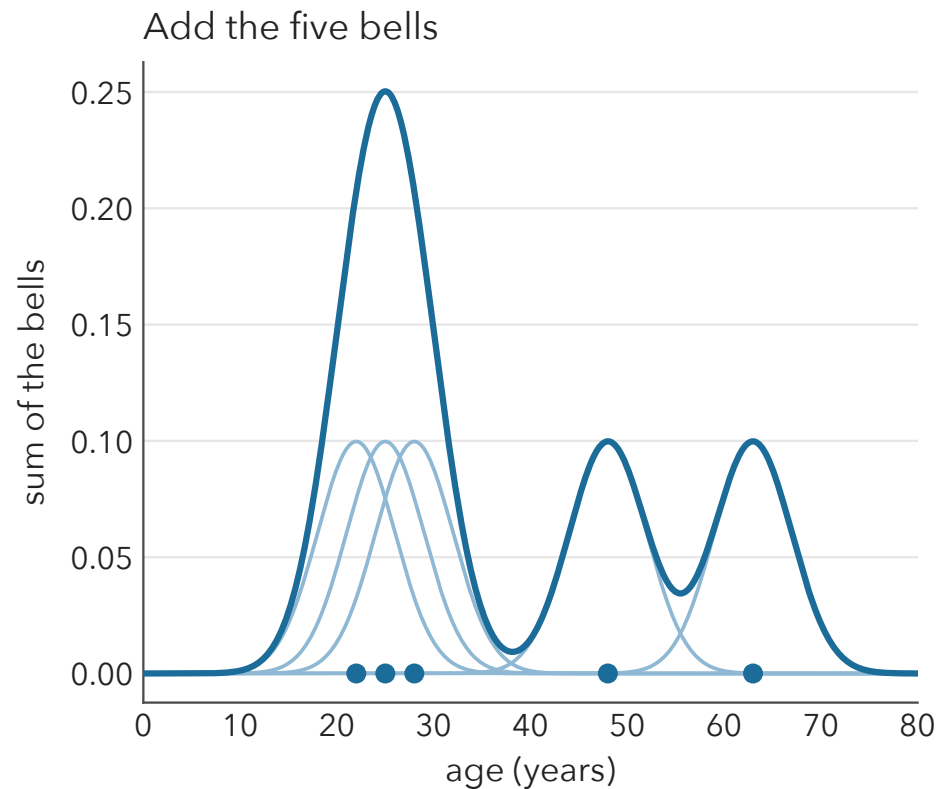
PART 2

Density curves

Replace each passenger with a small bell. Add the bells, then divide by the number of people. The area under that curve is 1.

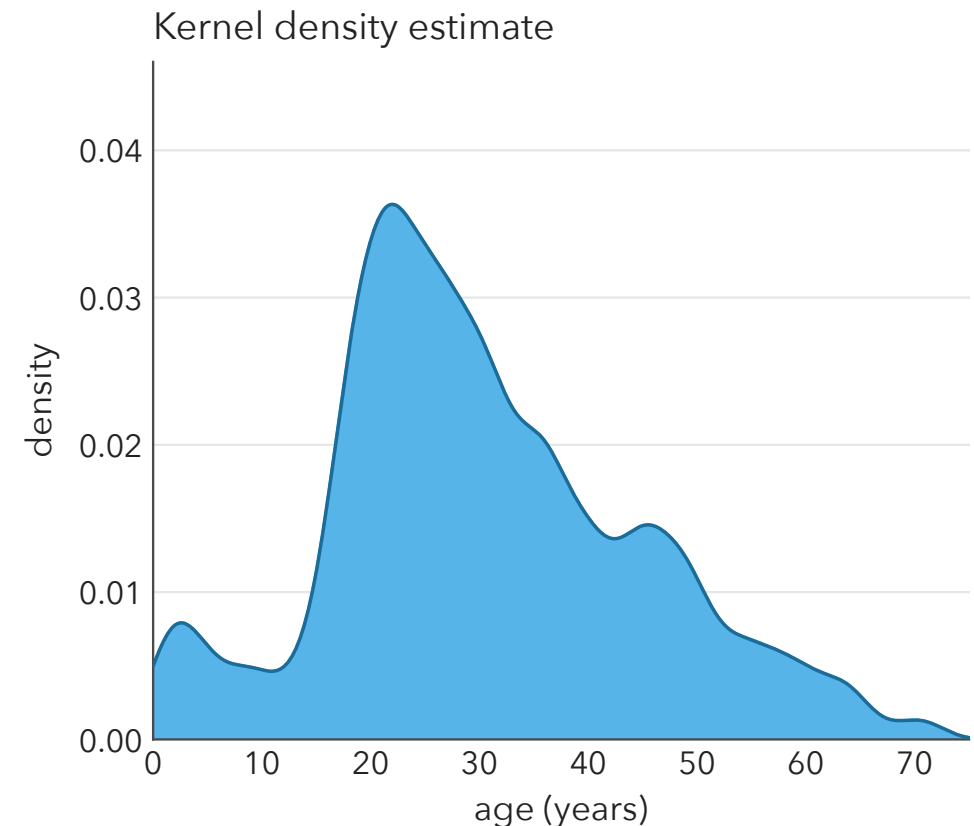
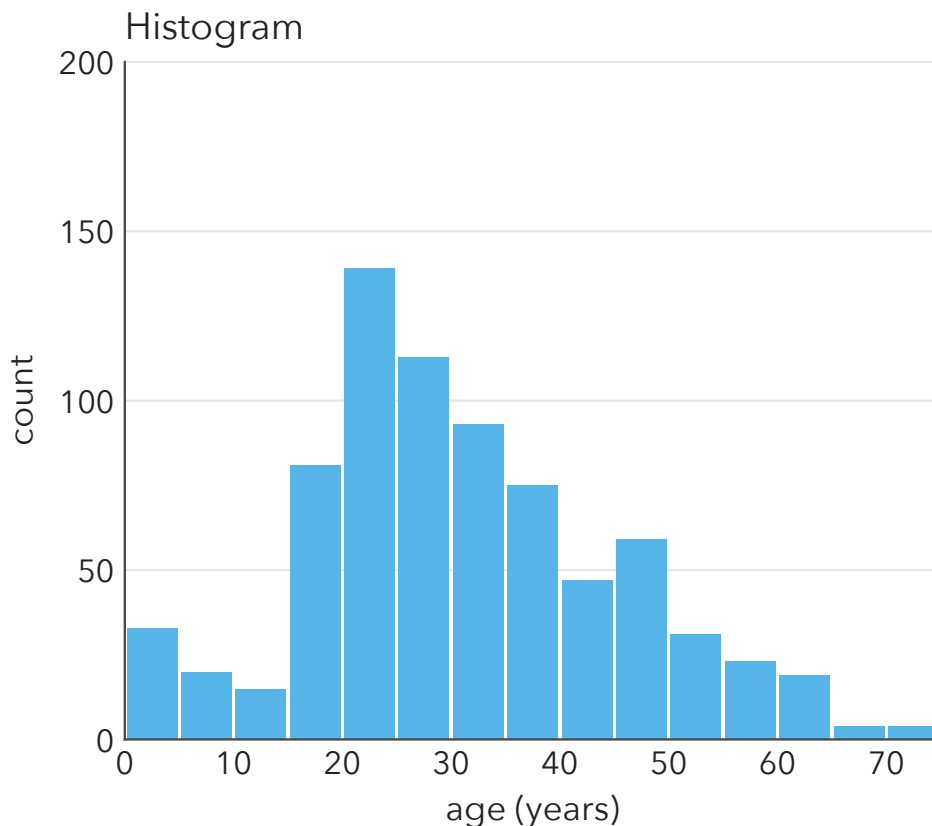
Five bells, then their average

These five ages are 22, 25, 28, 48, and 63. Each dot is one person. Each pale curve is that person's bell, with bandwidth 4, so most of the bell sits within about 4 years of the dot. The three close ages pile up. Dividing the sum by 5 makes the area equal to 1. That curve is the density.



A histogram counts. A density is a proportion.

The histogram answers “how many people?” The density answers “what share of the people?” Because the area under the density is 1, the whole curve is everyone. This density uses a Gaussian bump with bandwidth 2, which means each bell is about 2 years wide.



Why the peak is about 0.037, not 139

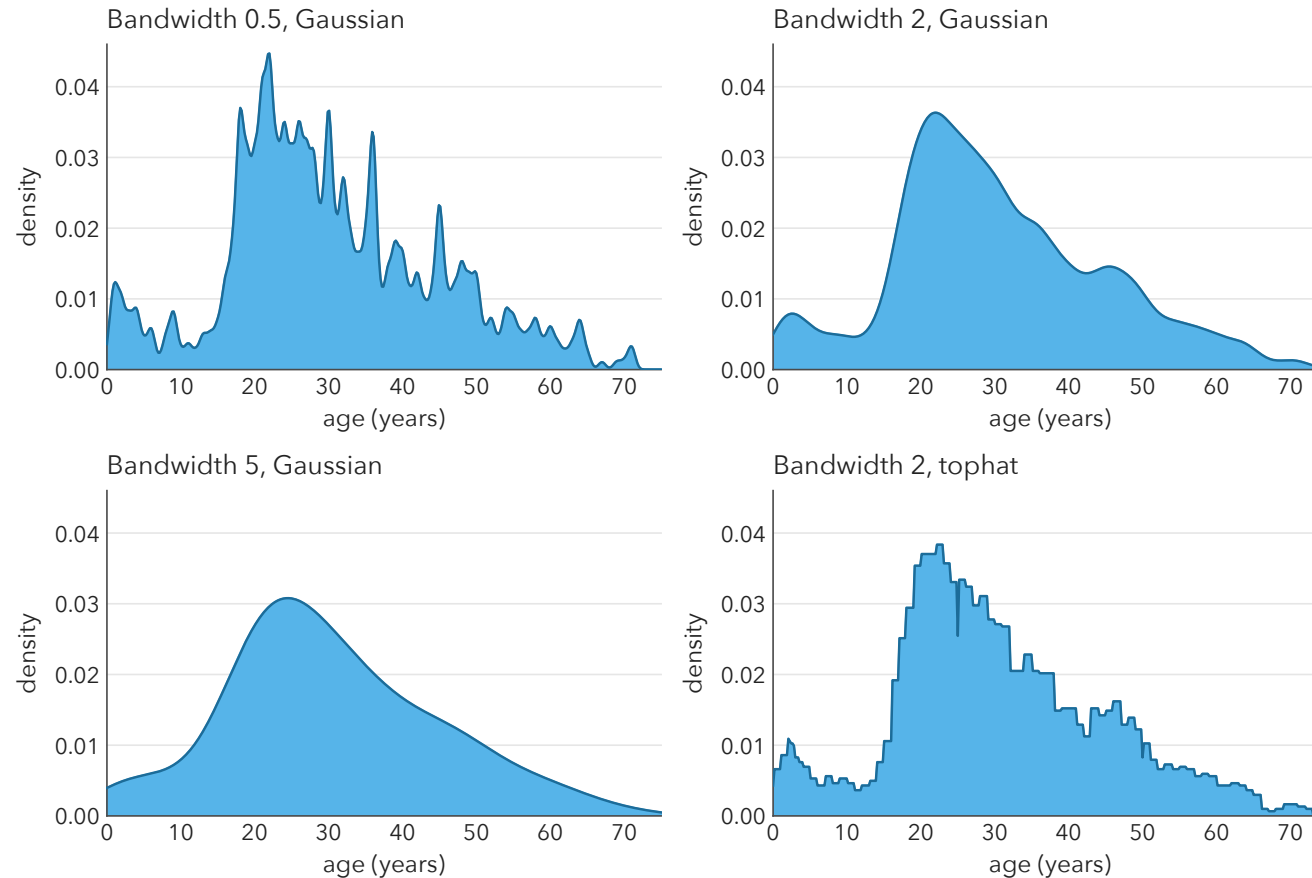
A histogram bar and a density are the same passengers on two scales. Divide the count by the number of people, then divide by the bin width. What remains is a share per year, which is the height of the density.

$$139 / (756 \times 5) \approx 0.037$$

139 is the tallest bar. 756 is the number of passengers. 5 is the width of that bar, in years. The density drawing tops out near 0.04 for this reason. It is not a count of people.

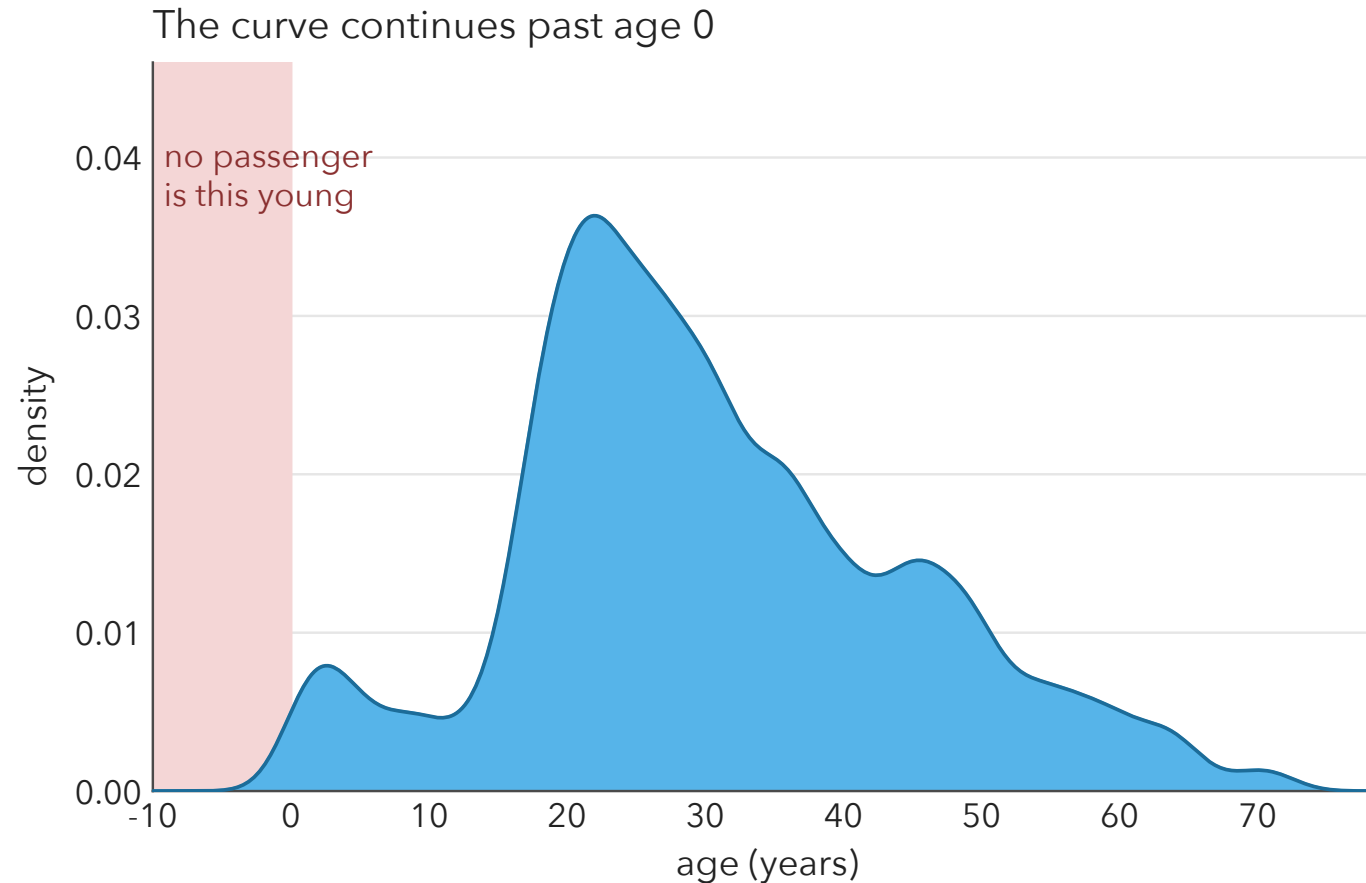
The curve depends on the bandwidth and the kernel

Bandwidth is the width of each passenger's bump, in years. A small bandwidth follows the noise. A large one washes the peak out. The kernel is the shape of the bump: a bell, called Gaussian, or a flat rectangle. scikit-learn calls the flat one **tophat**.



The curve can enter ages that never happen

The youngest passenger is 0.17 years old. The bumps centered on the infants still spread to the left, so the drawn curve includes negative ages. No passenger is younger than zero. That part of the curve is the kernel, not the data.



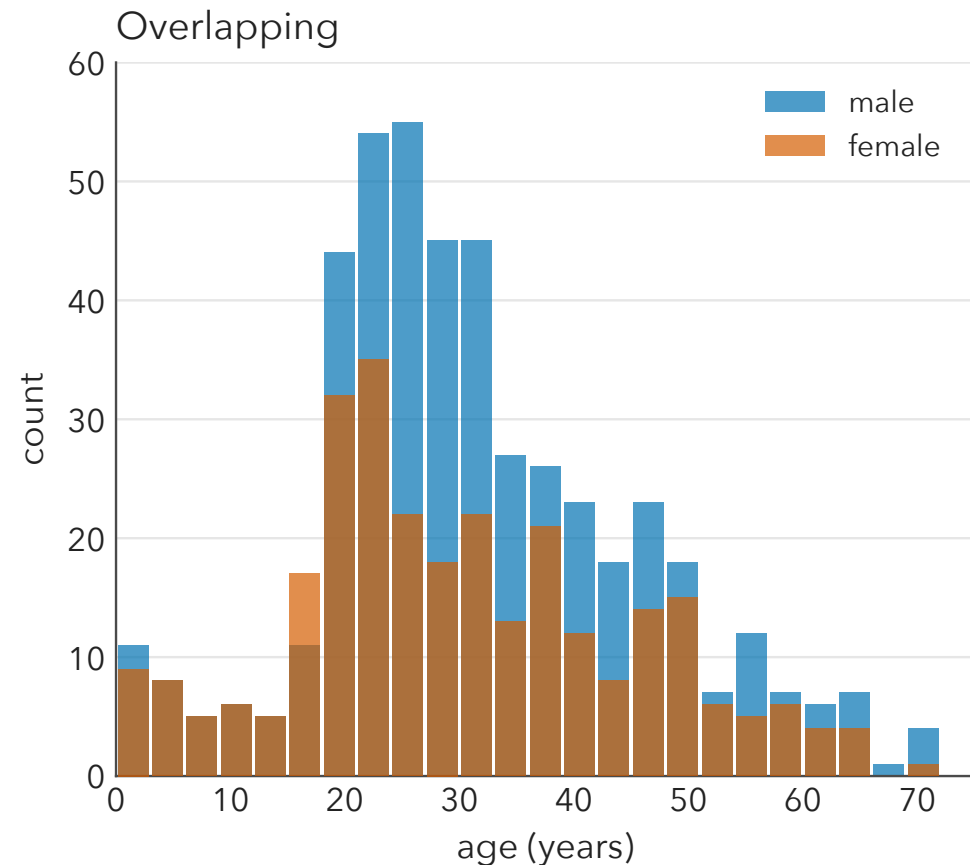
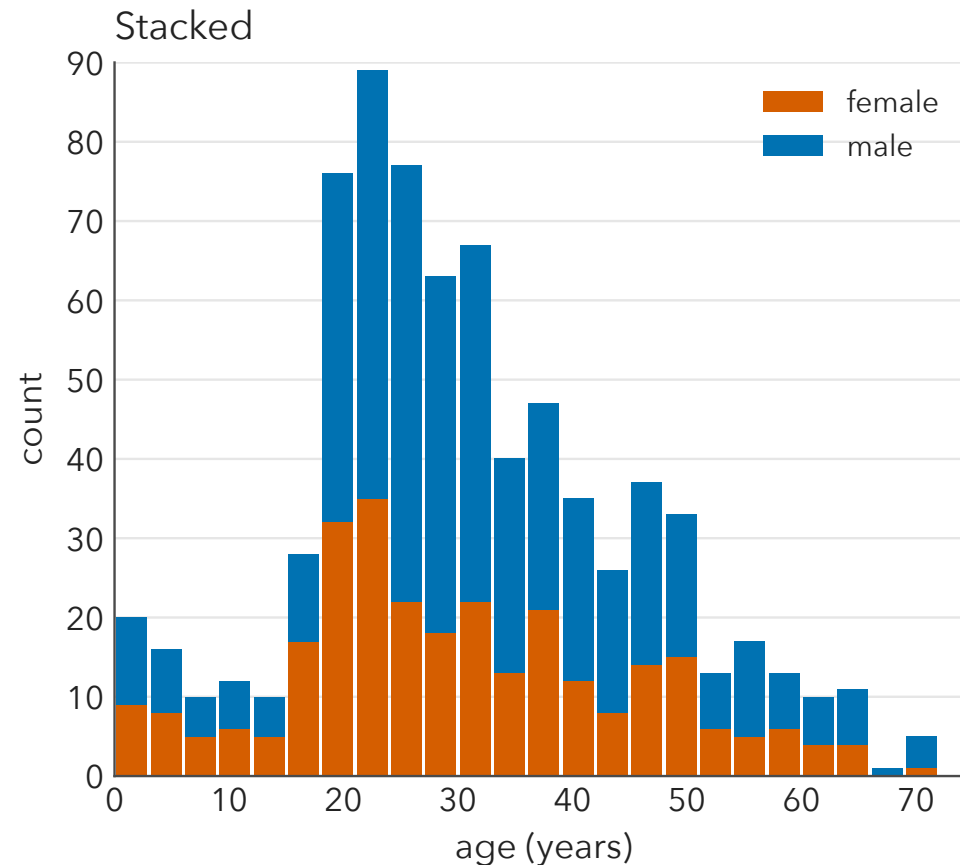
PART 3

Two groups at once

Men and women do not have the same age pattern. The picture has to keep those two counts separate.

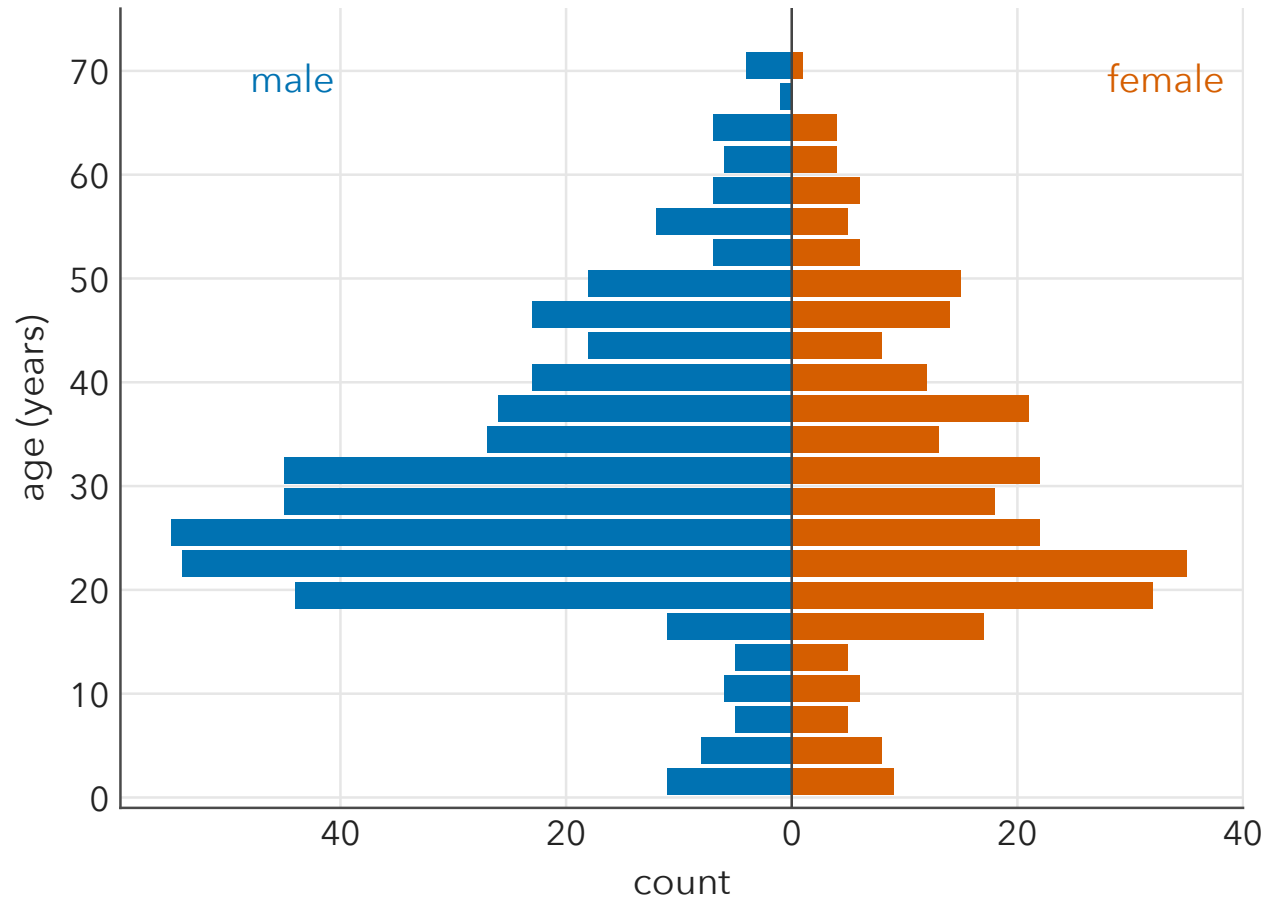
Stacking and overlapping are hard to read

On the left, the male bars sit on top of the female bars, so the male baseline moves. On the right, the bars cover each other. Either picture can hide which group is larger in a bin. These are rarely a good choice.



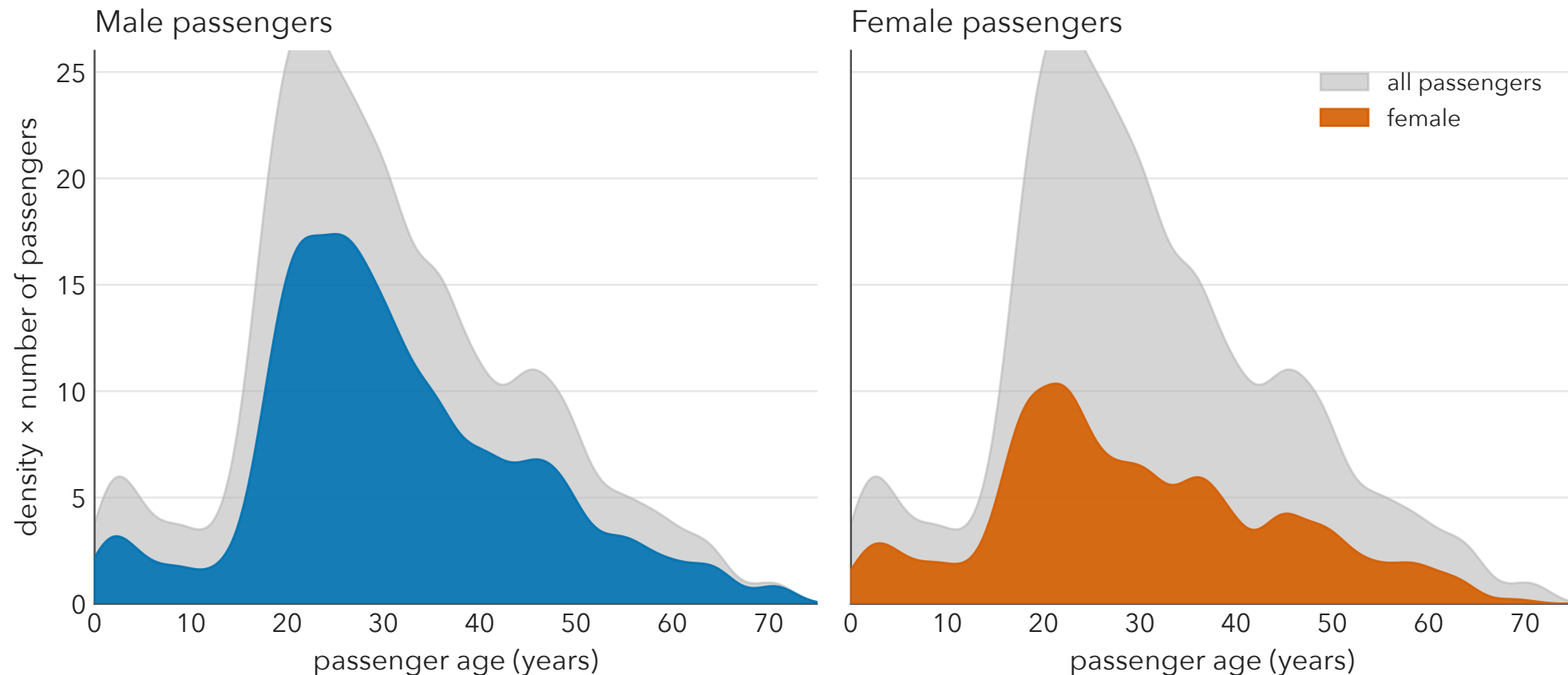
An age pyramid gives each group its own side

Male counts go to the left and female counts go to the right. They share the age axis, and both sides are measured from zero. There are 468 men and 288 women in the table, which is why the left side is longer.



Let the area follow the number of people

The gray shape is all 756 passengers. The colored shape is one sex. The curves are densities multiplied by the number of people, so the male area is larger because there are more men. A density that is forced to have area 1 in each panel would make the two groups look the same size.



PART 4

The same choices in matplotlib

pandas holds the table. numpy sets the bin edges. scikit-learn computes the density. matplotlib draws the result.

Read the table

```
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

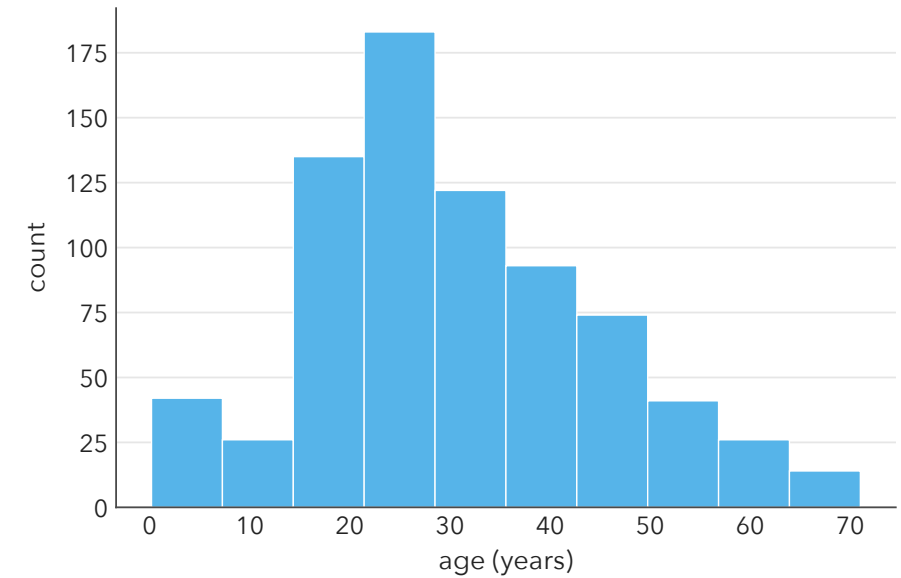
titanic = pd.read_csv("titanic.csv")
titanic = titanic[["age", "sex", "class", "survived"]]
```

`read_csv` turns the file into a table. The second line keeps the four columns this lecture uses.

Age is already in years. There is no missing age in this file, so every row can go into the histogram.

hist() will choose the bins if you do not

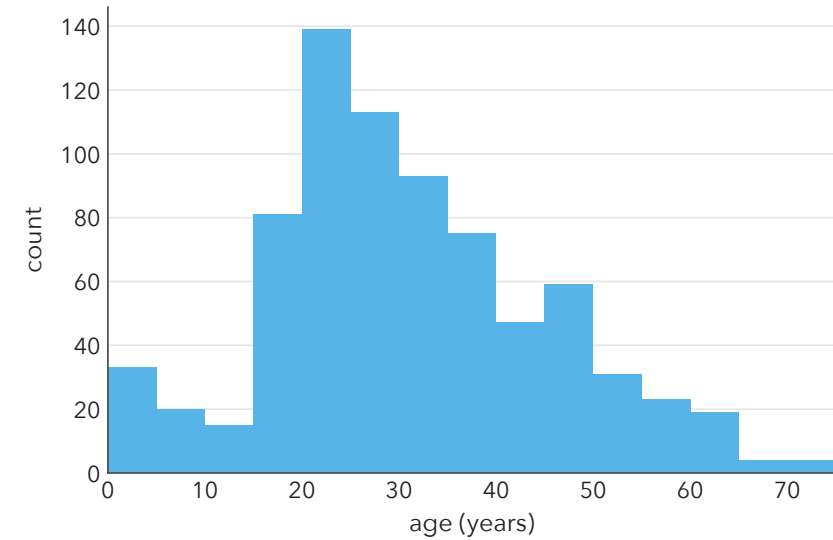
```
fig, ax = plt.subplots()
ax.hist(titanic["age"], color="#56B4E9")
ax.set_xlabel("age (years)")
ax.set_ylabel("count")
```



`hist` counts and draws in one step. The default is 10 bins, spread across the ages that happen to be in the column.

Hand hist the edges

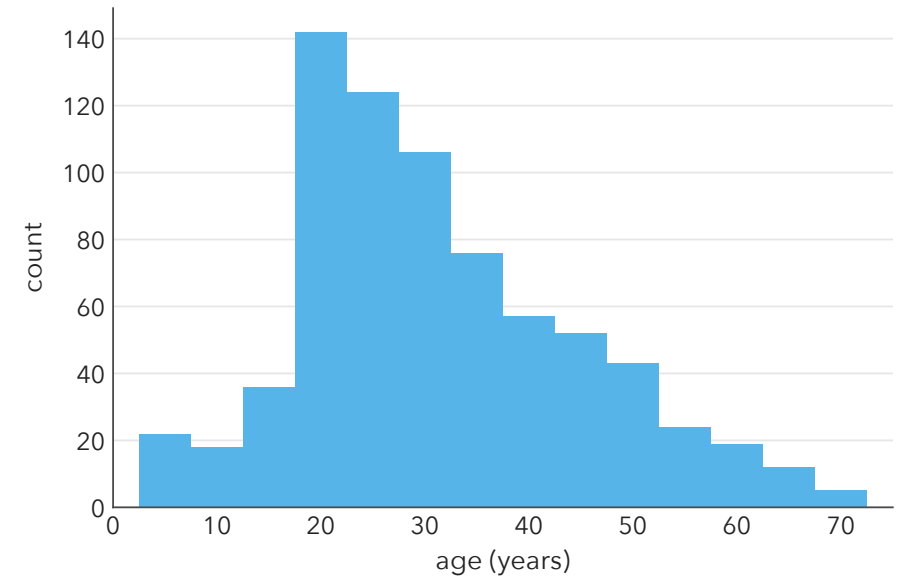
```
fig, ax = plt.subplots()
ax.hist(
    titanic["age"],
    bins=np.arange(0, 81, 5),
    color="#56B4E9",
)
ax.set_xlabel("age (years)")
ax.set_ylabel("count")
```



`np.arange(0, 81, 5)` is the list 0, 5, 10, ..., 80. Each bin runs from one number to the next, so every bar is 5 years wide.

The same width, moved over

```
ax.hist(  
    titanic["age"],  
    bins=np.arange(2.5, 78, 5),  
    color="#56B4E9",  
)  
ax.set_xlabel("age (years)")  
ax.set_ylabel("count")
```



The bins are still 5 years wide. They now run 2.5–7.5, 7.5–12.5, and so on. Passengers who sit near an edge change bars.

A density is a calculation, then a drawing

matplotlib draws the curve. scikit-learn computes it. `KernelDensity` wants each age in its own row, so `reshape(-1, 1)` makes that column. `bandwidth` is the width of the bell, in years. `score_samples` returns the log of the density, and `np.exp` turns that back into the density. The area under `y` is 1.

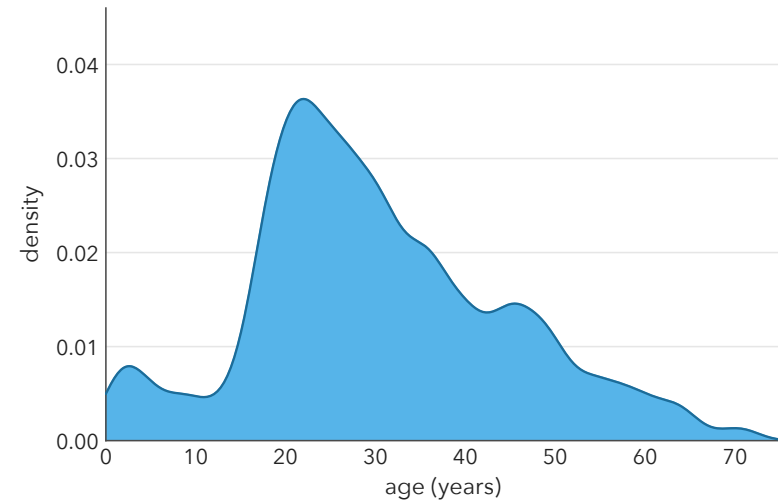
```
from sklearn.neighbors import KernelDensity

ages = titanic["age"].to_numpy().reshape(-1, 1)
grid = np.linspace(0, 75, 400).reshape(-1, 1)

kde = KernelDensity(kernel="gaussian", bandwidth=2)
kde.fit(ages)
y = np.exp(kde.score_samples(grid))
```

Fill under the curve

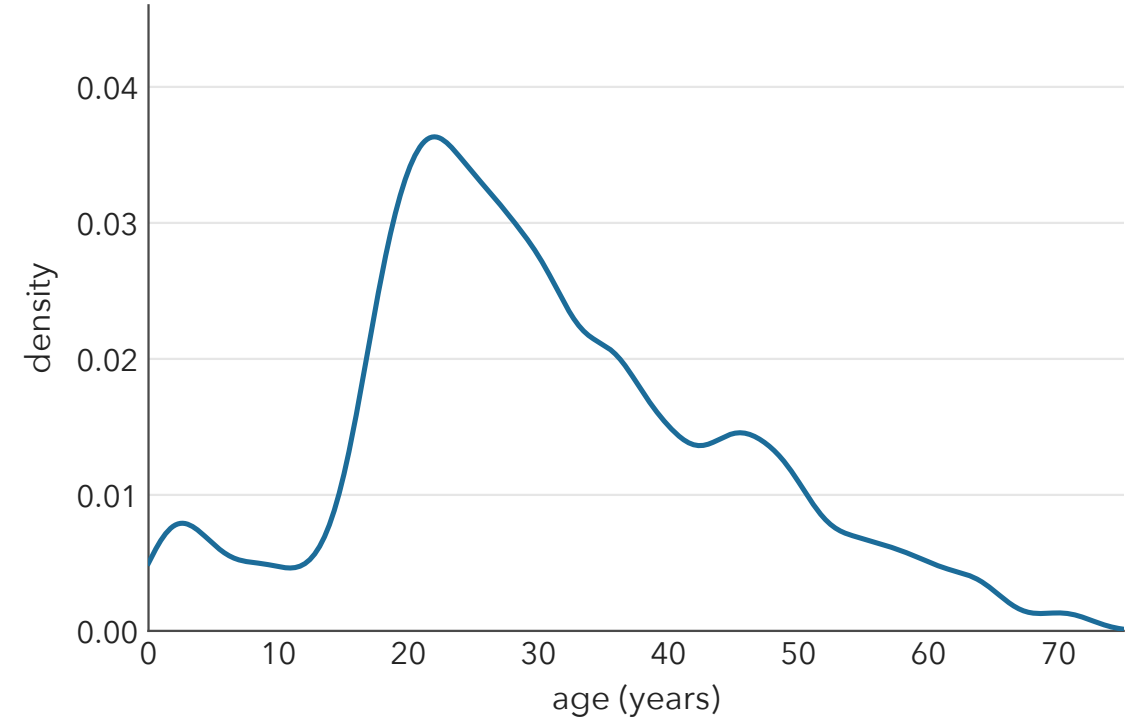
```
fig, ax = plt.subplots()
ax.fill_between(grid.ravel(), y, color="#56B4E9")
ax.plot(grid.ravel(), y, color="#1C6C99")
ax.set_xlim(0, 75)
ax.set_ylim(0, None)
ax.set_xlabel("age (years)")
ax.set_ylabel("density")
```



`fill_between` paints the area from 0 up to `y`. `plot` draws the edge of that same curve. `ravel()` turns the one-column grid back into a list of x positions.

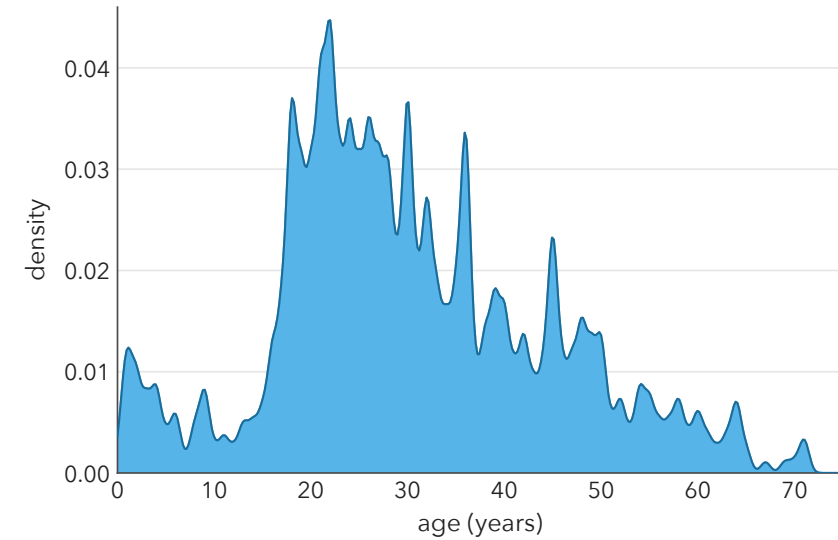
Or draw only the line

```
fig, ax = plt.subplots()
ax.plot(grid.ravel(), y, color="#1C6C99")
ax.set_xlim(0, 75)
ax.set_ylim(0, None)
ax.set_xlabel("age (years)")
ax.set_ylabel("density")
```



A smaller bandwidth keeps the small bumps

```
kde = KernelDensity(kernel="gaussian", bandwidth=0.5)
kde.fit(ages)
y = np.exp(kde.score_samples(grid))
ax.fill_between(grid.ravel(), y, color="#56B4E9")
ax.plot(grid.ravel(), y, color="#1C6C99")
ax.set_xlim(0, 75)
ax.set_ylim(0, 0.046)
```

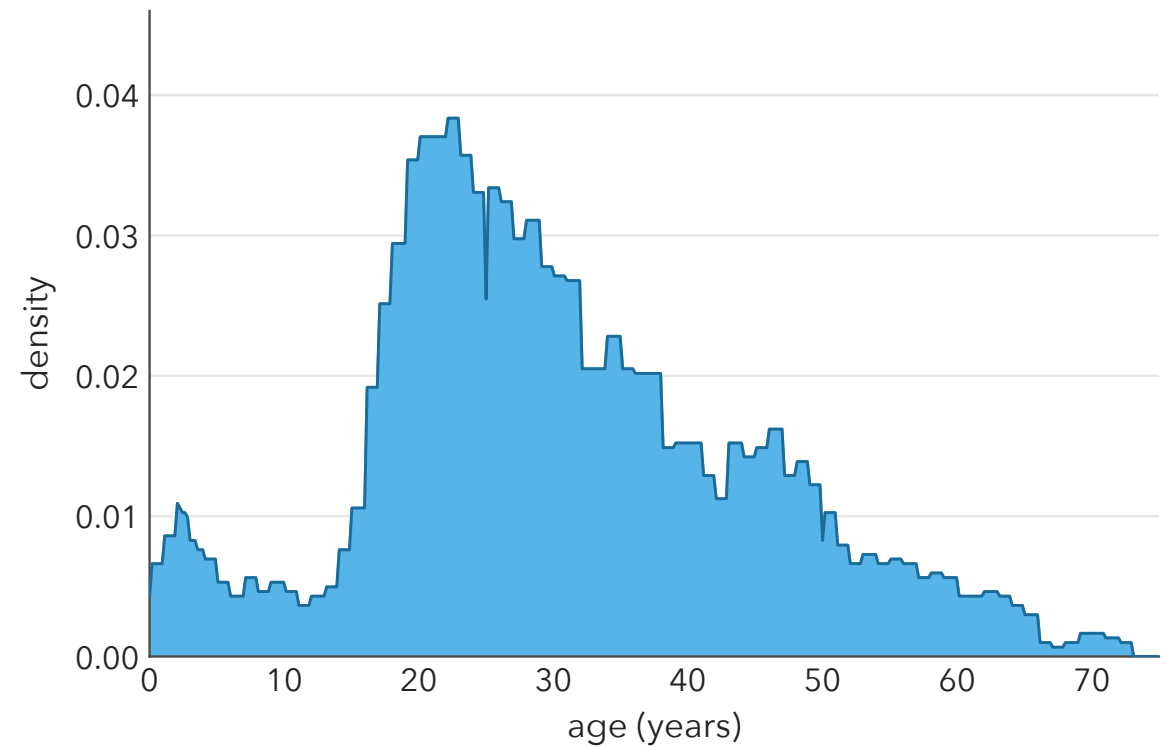


The vertical limit is the same as the bandwidth-2 curve. Several peaks now hit that ceiling. They are real clusters of a few ages, and also noise.

A tophat kernel is a flat bump

A tophat is flat. Each passenger contributes the same height between $\text{age} - \text{bandwidth}$ and $\text{age} + \text{bandwidth}$, and nothing outside that. The average is lumpier than the bell at the same bandwidth.

```
kde = KernelDensity(kernel="tophat", bandwidth=2)
kde.fit(ages)
y = np.exp(kde.score_samples(grid))
```



What the pictures were for

- **Histogram.** Edges plus counts. The height is how many cases fell in the bin, so the bars include zero.
- **Bin width.** Narrow bins follow noise. Wide bins hide a real group, such as the children.
- **Density.** One bell per person. Add the bells and divide by the number of people. The area is 1. Bandwidth is the width of the bell.
- **The link.** $\text{Count} \div (\text{number of people} \times \text{bin width})$ is about the density height. The tallest bar is $139 / (756 \times 5) \approx 0.037$, not 139.
- **Support.** The curve can enter values that never occur, such as a negative age. That part is the kernel, not the data.
- **Two groups.** A pyramid, or separate curves whose area follows the count. Stacked and overlapping bars are hard to read.

FURTHER READING

The chapter, and the two counting functions

- Claus O. Wilke, [Chapter 7: Histograms and density plots](#)
- matplotlib, [Axes.hist](#)
- scikit-learn, [KernelDensity](#)

Figures for Data 110, Fall 2026. Run `visualizing_distributions_1.py` to rebuild every figure.